Exercises Week 6 (**with solutions**)

Advanced Machine Learning (02460)
Technical University of Denmark
Søren Hauberg

March 2026
(Version 1.0)

1 Theoretical exercises

Exercise 6.1 (Quadratic metric) Consider a two-dimensional abstract manifold with the metric

$$\mathbf{G}_{\mathbf{x}} = \left(1 + \|\mathbf{x}\|^2\right)\mathbf{I}, \qquad \mathbf{x} \in \mathbb{R}^2. \tag{1}$$

Consider the points

$$\mathbf{x}_1 = \begin{pmatrix} 1 \\ 1 \end{pmatrix} \qquad \mathbf{x}_2 = \begin{pmatrix} 2 \\ 3 \end{pmatrix} \qquad \mathbf{x}_3 = \begin{pmatrix} 0 \\ 3 \end{pmatrix}. \tag{2}$$

1. *Compute the local norms of the tangent vector* $\mathbf{v}_1 = \begin{pmatrix} 1 & 0 \end{pmatrix}^\intercal$ *assuming the point of tangency is* $\mathbf{x}_1$, $\mathbf{x}_2$ *and* $\mathbf{x}_3$, *respectively.*
2. *Compute the local angles between* $\mathbf{v}_1$ *and* $\mathbf{v}_2 = \begin{pmatrix} 0 & 1 \end{pmatrix}^\intercal$ *in the same three points of tangency.*

Solution 6.1 For the first question, note that in general

$$\begin{aligned} \|\mathbf{v}\|_{\mathbf{G}_{\mathbf{x}}} &= \sqrt{\mathbf{v}^\intercal \mathbf{G}_{\mathbf{x}} \mathbf{v}} && (3)\\ &= \sqrt{\mathbf{v}^\intercal \left(1 + \|\mathbf{x}\|^2\right)\mathbf{I}\mathbf{v}} && (4)\\ &= \sqrt{1 + \|\mathbf{x}\|^2}\ \|\mathbf{v}\|. && (5) \end{aligned}$$

We then have

$$\begin{aligned} \|\mathbf{v}\|_{\mathbf{G}_{\mathbf{x}_1}} &= \sqrt{1+2}\|\mathbf{v}_1\| = \sqrt{3} && (6)\\ \|\mathbf{v}\|_{\mathbf{G}_{\mathbf{x}_2}} &= \sqrt{1+(4+9)} = \sqrt{14} && (7)\\ \|\mathbf{v}\|_{\mathbf{G}_{\mathbf{x}_3}} &= \sqrt{1+(0+9)} = \sqrt{10}. && (8) \end{aligned}$$

For the second question, note that in general

$$\angle(\mathbf{v}_1, \mathbf{v}_2)_{\mathbf{G}_\mathbf{x}} = \cos^{-1}\left(\frac{\langle \mathbf{v}_1, \mathbf{v}_2\rangle_{\mathbf{G}_\mathbf{x}}}{\|\mathbf{v}_1\|_{\mathbf{G}_\mathbf{x}}\ \|\mathbf{v}_2\|_{\mathbf{G}_\mathbf{x}}}\right) \tag{9}$$

$$= \cos^{-1}\left(\frac{(1+\|\mathbf{x}\|^2)\mathbf{v}_1^\intercal \mathbf{v}_2}{\sqrt{1+\|\mathbf{x}\|^2}\|\mathbf{v}_1\|\ \sqrt{1+\|\mathbf{x}\|^2}\|\mathbf{v}_2\|}\right) \tag{10}$$

$$= \cos^{-1}\left(\frac{\mathbf{v}_1^\intercal \mathbf{v}_2}{\|\mathbf{v}_1\|\ \|\mathbf{v}_2\|}\right) \tag{11}$$

$$= \angle(\mathbf{v}_1, \mathbf{v}_2). \tag{12}$$

We, thus, have that in all three cases, the angle is 90°.

Exercise 6.2 (Euclidean metric) Consider the Euclidean metric of $\mathbb{R}^d$, i.e. $\mathbf{G} = \mathbf{I}$.

1. *Derive the coefficients of the geodesics* ODE *of this metric (Eq. 7.20 in the LMLG book).*
2. *Derive the geodesic* ODE.
3. *What is the geodesic that connects points* $\mathbf{x}_1$ *and* $\mathbf{x}_2$*?*

Solution 6.2 For the first question, write out the coefficients as

$$\Gamma_j^{ik}(c_t) = \frac{1}{2}\left[\mathbf{G}_{c_t}^{-1}\right]_{ij}\left(2\frac{\partial\,[G_{c_t}]_{ij}}{\partial\,[c_t]_k} - \frac{\partial\,[G_{c_t}]_{jk}}{\partial\,[c_t]_i}\right) \tag{13}$$

$$= \frac{1}{2}\,[\mathbf{I}]_{ij}\left(2\underbrace{\frac{\partial\,[\mathbf{I}]_{ij}}{\partial\,[c_t]_k}}_{=0} - \underbrace{\frac{\partial\,[\mathbf{I}]_{jk}}{\partial\,[c_t]_i}}_{=0}\right) = 0. \tag{14}$$

For the second question, we insert the zero-coefficients into the geodesic ODE

$$[\ddot{c}_t]_j = -\sum_{i=1}^{d}\sum_{k=1}^{d}\Gamma_j^{ik}(c_t)\,[\dot{c}_t]_k\,[\dot{c}_t]_j \tag{15}$$

$$= -\sum_{i=1}^{d}\sum_{k=1}^{d} 0\,[\dot{c}_t]_k\,[\dot{c}_t]_j = 0. \tag{16}$$

The geodesic ODE, thus, reduces to $\ddot{c}_t = \mathbf{0}$.

For the third question, we note that the only curve with second derivative everywhere is the straight line. Hence the geodesic is

$$c_t = (1-t)\mathbf{x}_1 + t\mathbf{x}_2. \tag{17}$$

Exercise 6.3 (Quadratic metric) Consider a two-dimensional abstract manifold with the metric

$$\mathbf{G}_{\mathbf{x}} = \left(1 + \|\mathbf{x}\|^2\right)\mathbf{I}, \qquad \mathbf{x} \in \mathbb{R}^2. \tag{18}$$

1. *Derive the coefficients of the geodesics* ODE *of this metric (Eq. 7.20 in the LMLG book). Hint: note that some terms of the metric are zero, which renders some coefficients to be zero as well.*
2. *Derive the geodesic* ODE.
3. *Consider a geodesic c starting at $c_0 = \mathbf{0}$ and initial velocity $\dot{c}_0 = \mathbf{v}$. What is the acceleration $\ddot{c}_0$?*

Solution 6.3 For the first question, we note that

$$\frac{\partial \mathbf{G}_{\mathbf{x}}}{\partial x_i} = \frac{\partial\left(1 + \|\mathbf{x}\|^2\right)}{\partial x_i}\mathbf{I} = 2x_i\mathbf{I}. \tag{19}$$

We then get the general coefficients

$$\Gamma_j^{ik} = \frac{1}{1 + \|\mathbf{x}\|^2}\left[\mathbf{I}\right]_{ij}\left(2x_k[\mathbf{I}]_{ij} - x_i[\mathbf{I}]_{jk}\right). \tag{20}$$

From this we see that $\Gamma_j^{ik} = 0$ for $i \neq j$. For other values, we get

$$\Gamma_1^{11} = \frac{x_1}{1 + \|\mathbf{x}\|^2} \tag{21}$$

$$\Gamma_1^{12} = \frac{2x_2}{1 + \|\mathbf{x}\|^2} \tag{22}$$

$$\Gamma_2^{21} = \frac{2x_1}{1 + \|\mathbf{x}\|^2} \tag{23}$$

$$\Gamma_2^{22} = \frac{x_2}{1 + \|\mathbf{x}\|^2}. \tag{24}$$

For the second question we simply insert the coefficients into the geodesic ODE

$$[\ddot{c}_t]_j = -\sum_{i=1}^{2}\sum_{k=1}^{2}\Gamma_j^{ik}[\dot{c}_t]_k[\dot{c}_t]_j \tag{25}$$

$$= -\sum_{k=1}^{2}\Gamma_j^{jk}[\dot{c}_t]_k[\dot{c}_t]_j \tag{26}$$

$$= -[\dot{c}_t]_j\sum_{k=1}^{2}\Gamma_j^{jk}[\dot{c}_t]_k \tag{27}$$

$$= -[\dot{c}_t]_j\begin{pmatrix}\Gamma_j^{j1}\\ \Gamma_j^{j2}\end{pmatrix}^{\mathsf{T}}\dot{c}_t. \tag{28}$$

We can then write this out as

$$\ddot{c}_t = -\dot{c}_t \circ \left(\begin{pmatrix} \Gamma_1^{11} & \Gamma_1^{12} \\ \Gamma_2^{21} & \Gamma_2^{22} \end{pmatrix} \dot{c}_t \right) \tag{29}$$

$$= -\frac{1}{1 + \|c_t\|^2} \dot{c}_t \circ \left(\begin{pmatrix} [c_t]_1 & 2[c_t]_2 \\ 2[c_t]_1 & [c_t]_2 \end{pmatrix} \dot{c}_t \right), \tag{30}$$

where $\circ$ denotes the Hadamard (elementwise) product.

For the third question, we insert to see that

$$\ddot{c}_{\mathbf{0}} = -\mathbf{v} \circ \left(\begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix} \mathbf{v} \right) = \mathbf{0}. \tag{31}$$

This implies that around $\mathbf{x} = \mathbf{0}$, geodesics are (locally) straight lines with direction $\mathbf{v}$.

2 Programming exercises

Exercise 6.4 (Curve parametrizations) Consider a two-dimensional abstract manifold with the metric

$$\mathbf{G}_{\mathbf{x}} = \left(1 + \|\mathbf{x}\|^2\right) \mathbf{I}, \qquad \mathbf{x} \in \mathbb{R}^2. \tag{32}$$

1. *Implement direct energy minimization for computing geodesics using piecewise straight lines to parameterize the solution curve.*
2. *Extend the previous implementation to also support third-order polynomials to parametrize the solution curve.*

Exercise 6.5 (Density metrics) Consider the dataset available at

https://www2.compute.dtu.dk/~sohau/weekendwithbernie/toybanana.npy

This consist of $N = 992$ observations in $\mathbb{R}^2$. Consider the metric over $\mathbb{R}^2$ defined as

$$\mathbf{G}_{\mathbf{x}} = \frac{1}{p(\mathbf{x}) + \epsilon} \tag{33}$$

$$p(\mathbf{x}) = \frac{1}{N} \sum_{n=1}^{N} \mathcal{N}(\mathbf{x} | \mathbf{x}_n, \sigma^2 \mathbf{I}), \tag{34}$$

where $\sigma = 0.1$, and $\epsilon = 10^{-4}$ avoids dividing by zero.

1. *Implement direct energy minimization for computing geodesics using piecewise straight lines to parameterize the solution curve.*

2. *Extend the previous implementation to also support third-order polynomials to parametrize the solution curve.*

Solution 6.5 The following code solves the two exercises using piecewise linear curves as geodesics. A polynomial implementation works similarly.

```python
import numpy as np
import torch
import torch.optim as optim
import matplotlib.pyplot as plt

def G1(x):
    """
    Evaluate the metric G(x) = (1 + |x|^2)I for torch.tensor x of shape Nx2.
    The result has shape Nx2x2
    """
    N, D = x.shape
    I = torch.eye(D, D).reshape(1, D, D).repeat(N, 1, 1)  # NxDxD
    alpha = 1 + torch.sum(x**2, dim=1)  # N
    G = alpha.reshape(N, 1, 1) * I  # NxDxD
    return G

def G2(x, data, sigma=0.1):
    """
    Evaluate the metric 1/p(x) * I, where p(x) is a Gaussian kernel density estimate.
    The result has shape Nx2x2
    """
    N, D = x.shape
    M, D = data.shape
    sigma2 = sigma**2
    normalization = (2 * 3.14159)**(D/2) * sigma**D  # scalar
    I = torch.eye(D, D).reshape(1, D, D)  # 1xDxD
    Gs = []
    for n in range(N):
        xn = x[n].reshape(1, D)  # 1xD
        delta = (xn - data)  # MxD
        K = torch.exp(-0.5 * torch.sum(delta**2, dim=1) / sigma2) / normalization # M
        pn = K.sum() / M  # scalar
        Gs.append(I / (pn + 1e-4))  # list containing 1xDxD matrices
    G = torch.concatenate(Gs, dim=0)  # NxDxD
    return G

def plot_metric(metric, range):
    X, Y = torch.meshgrid((range, range))
    XY = torch.concatenate((X.reshape(-1, 1), Y.reshape(-1, 1)), dim=1)
    G = metric(XY)  # NxDxD
    trG = G[:, 0, 0] + G[:, 1, 1]  # N
    plt.imshow(trG.reshape(X.shape).detach().numpy().T,
               extent=(range[0], range[-1], range[0], range[-1]),
               origin="lower")

```

```python
class PLcurve:
    def __init__(self, x0, x1, N):
        """
        Represent the piecewise linear curve connecting x0 to x1 using a total of N nodes (including end-points)
        """
        super(PLcurve, self).__init__()
        self.x0 = x0.reshape(1, -1)  # 1xD
        self.x1 = x1.reshape(1, -1)  # 1xD
        self.N = N

        t = torch.linspace(0, 1, N).reshape(N, 1)  # Nx1
        c = (1-t) @ self.x0 + t @ self.x1  # NxD
        self.params = c[1:-1]  # (N-2)xD
        self.params.requires_grad = True

    def points(self):
        c = torch.concatenate((self.x0, self.params, self.x1), axis=0)  # NxD
        return c

    def plot(self):
        c = self.points().detach().numpy()
        plt.plot(c[:, 0], c[:, 1])


def curve_energy(metric, curve):
    G = metric(curve[:-1])  # (N-1)xDxD
    delta = curve[1:] - curve[:-1]  # (N-1)xD
    tmp = torch.bmm(G, delta.unsqueeze(-1)).squeeze(-1)  # (N-1)xD
    energy = torch.sum(delta * tmp)
    return energy

def connecting_geodesic(metric, curve):
    opt = optim.LBFGS([curve.params], lr=0.5)
    def closure():
        opt.zero_grad()
        energy = curve_energy(metric, curve.points())
        energy.backward()
        return energy

    max_iter = 1000
    for iter in range(max_iter):
        opt.zero_grad()
        opt.step(closure)


# metric = 'quadratic'
metric = 'density'
if metric == 'quadratic':
    r = 5
    plot_metric(G1, torch.linspace(-r, r, 100))
    N = 20
```

```
    for _ in range(5):
        c = PLcurve(2*r*(torch.rand(2)-0.5), 2*r*(torch.rand(2)-0.5), N)
        c.plot()
        print('Energy before optimization is {}'.format(curve_energy(G1, c.points()).item()))
        connecting_geodesic(G1, c)
        print('Energy after optimization is {}'.format(curve_energy(G1, c.points()).item()))
        c.plot()

    plt.axis((-r, r, -r, r))
    plt.show()

elif metric == 'density':
    data = torch.from_numpy(np.load('toybanana.npy')).to(torch.float32) # 992x2
    r = 3
    G = lambda x: G2(x, data)
    plot_metric(G, torch.linspace(-r, r, 100))
    plt.scatter(data[:, 0], data[:, 1], s=1)
    T = 20
    for _ in range(10):
        idx = torch.randint(data.shape[0], (2,))
        c = PLcurve(data[idx[0]], data[idx[1]], T)
        #c.plot()
        print('Energy before optimization is {}'.format(curve_energy(G, c.points()).item()))
        connecting_geodesic(G, c)
        print('Energy after optimization is {}'.format(curve_energy(G, c.points()).item()))
        c.plot()

    plt.axis((-r, r, -r, r))
    plt.show()
```